

BankManagement

Backend Architecture and Database Communication

Technical Implementation Report

Scope

Node.js/Express request processing, SQL Server connectivity, connection pools, direct stored-procedure execution, report-view queries, authentication, authorization, session handling, auditing, date handling, error translation, SQL-injection control, and proof that the implementation does not use an ORM.

Technology

Node.js, Express, the `mssql` driver, Microsoft SQL Server, T-SQL stored procedures and controlled reporting views.

Student: _____

Course: Database Systems

Instructor: _____

Date: _____

Executive Summary

The BankManagement backend is a REST API that mediates between a browser-based frontend and a Microsoft SQL Server database. The backend does not duplicate the database business logic and does not map tables to application model classes. Instead, it performs HTTP-specific work—routing, request validation, CORS, rate limiting, bearer-token extraction, broad role checks, paging, response formatting, and error translation—then calls explicit SQL Server stored procedures or a small set of controlled reporting views through the `mssql` driver.

The database remains authoritative for credentials, server-side sessions, effective roles, branch assignments, ownership, account and loan rules, transactional consistency, locking, audit records, triggers, and scheduled maintenance. Every protected HTTP request first executes `dbo.sp_User_ValidateSession`; the returned identity context is attached to the Express request as `req.user`. Business routes pass the authenticated `UserID`, never a client-trusted identity, to the relevant stored procedure.

The integrated implementation contains 21 backend JavaScript files (approximately 2,091 source lines), 56 HTTP endpoints including health diagnostics, four named SQL connection pools, 50 stored procedures called directly from Node.js, and 14 whitelisted API reports. SQL values are parameterized with explicit SQL Server types. Dynamic report identifiers are not accepted from the client; report keys select a server-owned configuration entry containing a fixed view name and fixed ordering expression.

System path

Browser → Express middleware → session validation → route authorization and access scope → typed `mssql` request → stored procedure or controlled view → SQL Server transaction/rules/audit → normalized JSON response.

Contents

Executive Summary	i
1 Purpose, Scope, and Terminology	1
1.1 Purpose	1
1.2 Scope of the implementation	1
1.3 Terminology	1
2 System Architecture	2
2.1 Layered design	2
2.2 Responsibility boundary	2
2.3 Runtime addresses	2
3 Backend Source Structure	3
3.1 Directory layout	3
3.2 Module roles	3
4 Startup and Shutdown Lifecycle	5
4.1 Application startup	5
4.2 Graceful shutdown	5
5 Global Express Request Pipeline	6
5.1 Middleware order	6
5.2 Health endpoint	6
6 Environment Configuration	7
6.1 Configuration loading	7
6.2 Important variables	7
7 SQL Server Connection Pools	8
7.1 Why multiple pools exist	8
7.2 Pool configuration	8
7.3 Lazy connection and caching	8
7.4 Communication protocol	9
8 Direct Data-Access Layer	10
8.1 Explicit SQL type map	10
8.2 Input binding	10
8.3 Stored-procedure execution	10
8.4 Procedure results	11
8.5 Parameterized direct queries	11

9 End-to-End Request Lifecycle	12
9.1 Protected request sequence	12
9.2 Why identity is not accepted from the client	12
10 Authentication	14
10.1 Three separate authentication contexts	14
10.2 Signup	14
10.3 Login	14
10.4 Token design	15
10.5 Session validation	15
10.6 Creation of <code>req.user</code>	15
10.7 Logout	16
11 Authorization	17
11.1 Route-level role authorization	17
11.2 Owner, branch, and global scope	17
11.3 Account ownership	17
11.4 Employee scope	17
11.5 Loan scope	18
11.6 Defense in depth	18
12 Business Procedure Communication	19
12.1 Route-to-procedure pattern	19
12.2 Example: opening an account	19
12.3 Example: transfer	19
12.4 Example: loan status	20
13 Reporting Communication	21
13.1 Report catalogue	21
13.2 Safe dynamic SQL	21
13.3 Branch filtering strategies	21
13.4 Paging	22
14 Database-Level Security	23
14.1 Technical SQL principals	23
14.2 Database roles and privileges	23
14.3 Application roles are different	23
15 SQL Injection Control	24
15.1 Bound data values	24
15.2 Identifiers	24
15.3 Static loan query	24
15.4 Stored procedures are not sufficient by themselves	24
16 Proof That No ORM Is Used	25
16.1 Dependency evidence	25
16.2 Connection evidence	25
16.3 Execution evidence	25
16.4 Route evidence	25
16.5 Absent ORM artifacts	26

17 Date and Time Handling	27
17.1 SQL driver configuration	27
17.2 Date-only versus date-time	27
17.3 Response metadata	27
18 HTTP Responses and Error Handling	28
18.1 Success response	28
18.2 Errors	28
19 Transactions, Concurrency, and Atomicity	29
19.1 Backend versus database transactions	29
19.2 Pending financial transactions	29
19.3 Locks	29
19.4 Triggers	29
20 Audit and Branch Ledger	30
20.1 Audit ownership	30
20.2 Session audit	30
20.3 Branch ledger	30
20.4 Safe audit reporting	30
21 Scheduled Jobs and Manual Maintenance	31
21.1 SQL Server Agent	31
21.2 Manual API equivalents	31
22 Detailed Communication Scenarios	32
22.1 Scenario 1: login	32
22.2 Scenario 2: personal transfer	32
22.3 Scenario 3: manager views branch accounts	32
22.4 Scenario 4: HighAdmin audit report	32
22.5 Scenario 5: session expires	33
23 Testing and Diagnostics	34
23.1 Syntax checks	34
23.2 Health and connection diagnosis	34
23.3 Postman testing	34
23.4 Useful diagnostic distinctions	34
24 Security and Production Hardening	35
24.1 Existing controls	35
24.2 Recommended production improvements	35
25 Conclusion	36
A Complete API-to-SQL Inventory	37
B Report Catalogue	42
C Stored Procedures Called by the Backend	45

List of Tables

1.1	Important terms	1
2.1	Layer responsibilities	2
3.1	Key backend files	3
6.1	Backend environment variables	7
7.1	Named SQL pools	8
10.1	Authentication contexts	14
11.1	Access-scope modes	17
13.1	Report branch-filter modes	21
14.1	Database roles	23
18.1	HTTP error behavior	28
A.1	Complete endpoint communication inventory	38
B.1	Whitelisted API reports	43
C.1	Directly called stored procedures	45

List of Figures

2.1 Backend and database communication layers	2
9.1 Protected API request lifecycle	12
22.1 Login communication sequence	32

Chapter 1

Purpose, Scope, and Terminology

1.1 Purpose

This report documents how the backend operates and how it communicates with the BankManagement database. The focus is the software boundary between HTTP and SQL Server: what is performed in Node.js, what is delegated to T-SQL, how identities and permissions cross the boundary, and how data is safely returned to the frontend.

1.2 Scope of the implementation

The report describes the integrated backend after the account/loan ownership, branch visibility, employee scope, date handling, ten-minute session expiration, dropdown catalogue, branch overview, and manager-hiring corrections.

1.3 Terminology

Table 1.1: Important terms

Term	Meaning in this project
Application user	A person represented by <code>dbo.Users</code> , linked to an active <code>Customer</code> and optionally an <code>Employee</code> .
Application role	A business role in <code>dbo.Roles</code> / <code>dbo.UserRoles</code> : <code>Customer</code> , <code>Employee</code> , <code>Admin</code> , or <code>HighAdmin</code> .
Effective role	A role that is currently valid after checking user status, customer status, employee status, job title, admin flag, and identity consistency.
SQL Server login	A server-level technical principal such as <code>BankAppLogin</code> ; it authenticates the Node service, not the human end user.
Database user	The database principal mapped to a SQL login, such as <code>BankAppUser</code> .
Session token	An opaque random bearer value stored in <code>dbo.Sessions</code> ; it is not a JWT and contains no trusted role claims.
Connection pool	A reusable set of SQL Server connections managed by <code>mssql</code> .
Stored procedure	A named T-SQL program containing database validation, transaction management, locking, writes, and audit behavior.
Controlled view	A read-only reporting projection whose columns and joins are defined by the database.
ORM	A framework that maps tables/relations to application models and usually generates SQL. This project does not use one.

Chapter 2

System Architecture

2.1 Layered design

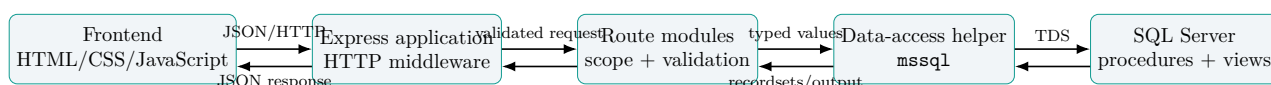


Figure 2.1: Backend and database communication layers

2.2 Responsibility boundary

Table 2.1: Layer responsibilities

Layer	Responsibilities
Frontend	Collect input, store the bearer token, call API endpoints, display role-specific sections, format date metadata, and render success/error messages. Frontend visibility is not authoritative security.
Backend	Receive HTTP requests, apply middleware, validate required fields, validate the session, attach <code>req.user</code> , determine broad role and scope, bind SQL types, select a connection pool, execute SQL, normalize dates, and translate results/errors into JSON.
Database	Verify passwords, create and expire sessions, calculate effective roles, enforce ownership and branch rules, validate entity state, use transactions and locks, update balances/installments, write audit records, fire triggers, and expose controlled reporting views.
SQL Server Agent	Execute trusted scheduled maintenance procedures without a browser session.

2.3 Runtime addresses

The normal local configuration uses:

- Frontend: `http://127.0.0.1:5500`
- Backend: `http://127.0.0.1:4000`
- SQL Server TCP: normally port 1433

Different browser ports are different origins, so CORS must explicitly permit the frontend origin. CORS controls browsers; it is not a substitute for authentication.

Chapter 3

Backend Source Structure

3.1 Directory layout

Listing 3.1: Backend source structure

```
1 src/
2   server.js
3   app.js
4   config/
5     env.js
6     db.js
7   middleware/
8     auth.js
9     validation.js
10  routes/
11    accounts.routes.js
12    auth.routes.js
13    branches.routes.js
14    customers.routes.js
15    employees.routes.js
16    highadmin.routes.js
17    loans.routes.js
18    reports.routes.js
19    setup.routes.js
20    transactions.routes.js
21    transfers.routes.js
22  utils/
23    access-scope.js
24    errors.js
25    http.js
26    procedure.js
```

3.2 Module roles

Table 3.1: Key backend files

File	Purpose
src/server.js	Starts the HTTP listener and performs graceful shutdown of the HTTP server and all database pools.
src/app.js	Constructs the Express application, global middleware, health route, route mounts, 404 handling, and central error handling.
src/config/env.js	Reads and validates environment variables, converts strings to booleans/numbers, and supplies defaults.

File	Purpose
<code>src/config/db.js</code>	Defines four SQL connection configurations, creates lazy pools, caches them, and closes them.
<code>src/middleware/auth.js</code>	Extracts bearer tokens, calls session validation, creates <code>req.user</code> , and applies broad application-role authorization.
<code>src/middleware/validation.js</code>	Rejects requests with missing required body fields.
<code>src/utils/procedure.js</code>	Maps JavaScript values to explicit SQL types and executes procedures or parameterized queries.
<code>src/utils/access-scope.js</code>	Resolves owner, branch, or global account/loan visibility and provides ownership assertions.
<code>src/utils/http.js</code>	Produces consistent JSON, paging metadata, and date/date-time normalization.
<code>src/utils/errors.js</code>	Converts SQL and application errors into structured HTTP responses.
<code>src/routes/*.routes.js</code>	Defines endpoint paths, required roles/fields, procedure parameters, and response metadata.

Chapter 4

Startup and Shutdown Lifecycle

4.1 Application startup

`src/server.js` imports the configured Express application and starts listening on the environment-defined port. A database connection is not opened eagerly at module load time; the first operation that asks for a named pool triggers connection creation.

Listing 4.1: HTTP server startup

```
1 const server = app.listen(env.port, () => {  
2   console.log(  
3     'BankManagement backend listening on http://localhost:${env.port}'  
4   );  
5 });
```

4.2 Graceful shutdown

The server listens for `SIGINT` and `SIGTERM`. It first stops accepting new HTTP requests, then closes fulfilled SQL pools, clears the pool cache, and exits. This avoids abandoning open sockets during normal development shutdown or service termination.

Shutdown order

Signal → `server.close()` → `closePools()` → close each connected pool → clear cache → process exit.

Chapter 5

Global Express Request Pipeline

5.1 Middleware order

The order configured in `src/app.js` is significant:

1. `helmet()` adds common HTTP security headers.
2. `cors(...)` checks the request origin against the configured allow-list.
3. `express.json({limit: '1mb'})` parses JSON and limits body size.
4. `morgan(...)` logs HTTP requests.
5. The health route tests the application database connection.
6. Functional routers are mounted under `/api/....`
7. The 404 handler catches unmatched routes.
8. The central error handler converts failures to JSON.

Listing 5.1: Global middleware and route mounting

```
1 app.use(helmet());
2 app.use(cors({ origin: configuredOriginCheck, credentials: true }));
3 app.use(express.json({ limit: '1mb' }));
4 app.use(morgan(env.nodeEnv === 'production' ? 'combined' : 'dev'));
5
6 app.use('/api/auth', authRoutes);
7 app.use('/api/accounts', accountRoutes);
8 app.use('/api/transactions', transactionRoutes);
9 app.use('/api/loans', loanRoutes);
10 app.use('/api/reports', reportRoutes);
11
12 app.use(notFound);
13 app.use(errorHandler);
```

5.2 Health endpoint

`GET/api/health` uses the application pool and runs a fixed diagnostic query:

```
1 SELECT
2   DB_NAME() AS databaseName,
3   SUSER_SNAME() AS sqlLogin,
4   USER_NAME() AS databaseUser;
```

This confirms that the backend is connected to the expected database through the expected SQL principal.

Chapter 6

Environment Configuration

6.1 Configuration loading

`src/config/env.js` loads `.env` through `dotenv`, converts textual booleans/numbers, and throws immediately when required credentials are absent. Failing at startup is preferable to silently connecting with an incomplete configuration.

6.2 Important variables

Table 6.1: Backend environment variables

Variable	Default	Purpose
PORT	4000	HTTP listening port.
NODE_ENV	development	Logging and stack-trace behavior.
CORS_ORIGIN	empty list	Comma-separated permitted frontend origins.
SESSION_TTL_MINUTES	10	Fixed session lifetime; clamped to 1–1440 minutes.
DB_SERVER, DB_PORT	localhost, 1433	SQL Server endpoint.
DB_DATABASE	BankManagement	Target database.
DB_USER, DB_PASSWORD	required	Main application SQL login.
DB_ENCRYPT	false	SQL transport encryption option.
DB_TRUST_SERVER_CERTIFICATE	true	Local certificate behavior.
DB_USE_UTC	false	Interpret SQL DATETIME as local SQL Server time.
DB_REQUEST_TIMEOUT_MS	30000	Per-request SQL timeout.
DB_CONNECTION_TIMEOUT_MS	15000	Connection establishment timeout.
DB_POOL_MAX, DB_POOL_MIN	10, 0	Pool size.
REPORT_DB_*	app fallback	Read-only reporting SQL identity.
AUDIT_DB_*	app fallback	Audit/ledger SQL identity.
HIGHADMIN_REPORT_DB_*	app fallback	HighAdmin reporting SQL identity.

Credential handling

The real `.env` file is not part of the clean package. Passwords are supplied locally and should not be committed. Falling back to the application credentials keeps development usable, but configuring the dedicated report/audit credentials preserves least privilege.

Chapter 7

SQL Server Connection Pools

7.1 Why multiple pools exist

The backend defines four pools:

Table 7.1: Named SQL pools

Pool	Expected SQL login	Use
app	BankAppLogin	Stored-procedure execution and limited operational views.
report	BankReportLogin	General operational and aggregate reporting views.
audit	BankAuditorLogin	Safe audit trail and branch ledger views.
highadminReport	BankHighAdminReportLogin	HighAdmin-only oversight views.

7.2 Pool configuration

Each pool receives the same server/database/timeouts and a different user/password pair. The options include `enableArithAbort: true` and the configured `useUTC` behavior.

Listing 7.1: Pool definitions

```
1 const poolDefinitions = {
2   app: makeConfig(env.db.user, env.db.password),
3   report: makeConfig(env.reportDb.user, env.reportDb.password),
4   audit: makeConfig(env.auditDb.user, env.auditDb.password),
5   highadminReport:
6     makeConfig(env.highAdminReportDb.user,
7               env.highAdminReportDb.password)
8 };
```

7.3 Lazy connection and caching

`getPool(name)` creates a `sql.ConnectionPool` only the first time that name is requested. The map stores the connection promise, so concurrent first requests share the same connection attempt. The pool is reused for later requests rather than opening a new TCP connection for every API call.

Listing 7.2: Lazy pool creation

```
1 if (!pools.has(name)) {
2   const pool = new sql.ConnectionPool(config);
3   pools.set(name, pool.connect());
4 }
5 return pools.get(name);
```

7.4 Communication protocol

The project imports `mssql`; its installed dependency stack includes `tedious`. The driver communicates with SQL Server using the Tabular Data Stream protocol. The browser never speaks TDS and never receives SQL credentials.

Chapter 8

Direct Data-Access Layer

8.1 Explicit SQL type map

`src/utils/procedure.js` defines the SQL types used by routes:

Listing 8.1: Selected SQL type mappings

```
1 const TYPES = {
2   int: sql.Int,
3   bit: sql.Bit,
4   money: sql.Decimal(18, 2),
5   percent: sql.Decimal(5, 2),
6   date: sql.Date,
7   datetime: sql.DateTime,
8   string50: sql.NVarChar(50),
9   string4000: sql.NVarChar(4000),
10  token: sql.NVarChar(200)
11 };
```

8.2 Input binding

An input definition may either use the same JavaScript/SQL name or map a SQL parameter to a camel-case request field:

```
1 inputs: {
2   UserID: TYPES.int,
3   AccountID: [TYPES.int, 'accountID'],
4   Amount: [TYPES.money, 'amount']
5 }
```

The helper calls `request.input(name, type, value)`. Undefined or empty-string values become SQL NULL; numeric zero and Boolean false are preserved.

8.3 Stored-procedure execution

Listing 8.2: Core procedure executor

```
1 async function executeProcedure(
2   procedureName,
3   { inputs = {}, outputs = {}, values = {}, poolName = 'app' } = {}
4 ) {
5   const pool = await getPool(poolName);
6   const request = pool.request();
7
8   addInputs(request, inputs, values);
9   addOutputs(request, outputs);
```

```
10
11   const result = await request.execute(procedureName);
12
13   return {
14     recordsets: result.recordsets || [],
15     recordset: result.recordset || [],
16     output: result.output || {},
17     rowsAffected: result.rowsAffected || []
18   };
19 }
```

8.4 Procedure results

SQL Server can return:

- a first recordset (`result.recordset`);
- multiple recordsets (`result.recordsets`);
- output parameters (`result.output`);
- rows-affected arrays.

Routes choose the shape required by the frontend. Loan status, for example, returns a loan summary and a separate installment recordset.

8.5 Parameterized direct queries

`executeQuery` is used only where a controlled view query is appropriate. It still binds every data value with `request.input`. It is used for loan listing and the reporting subsystem.

Chapter 9

End-to-End Request Lifecycle

9.1 Protected request sequence

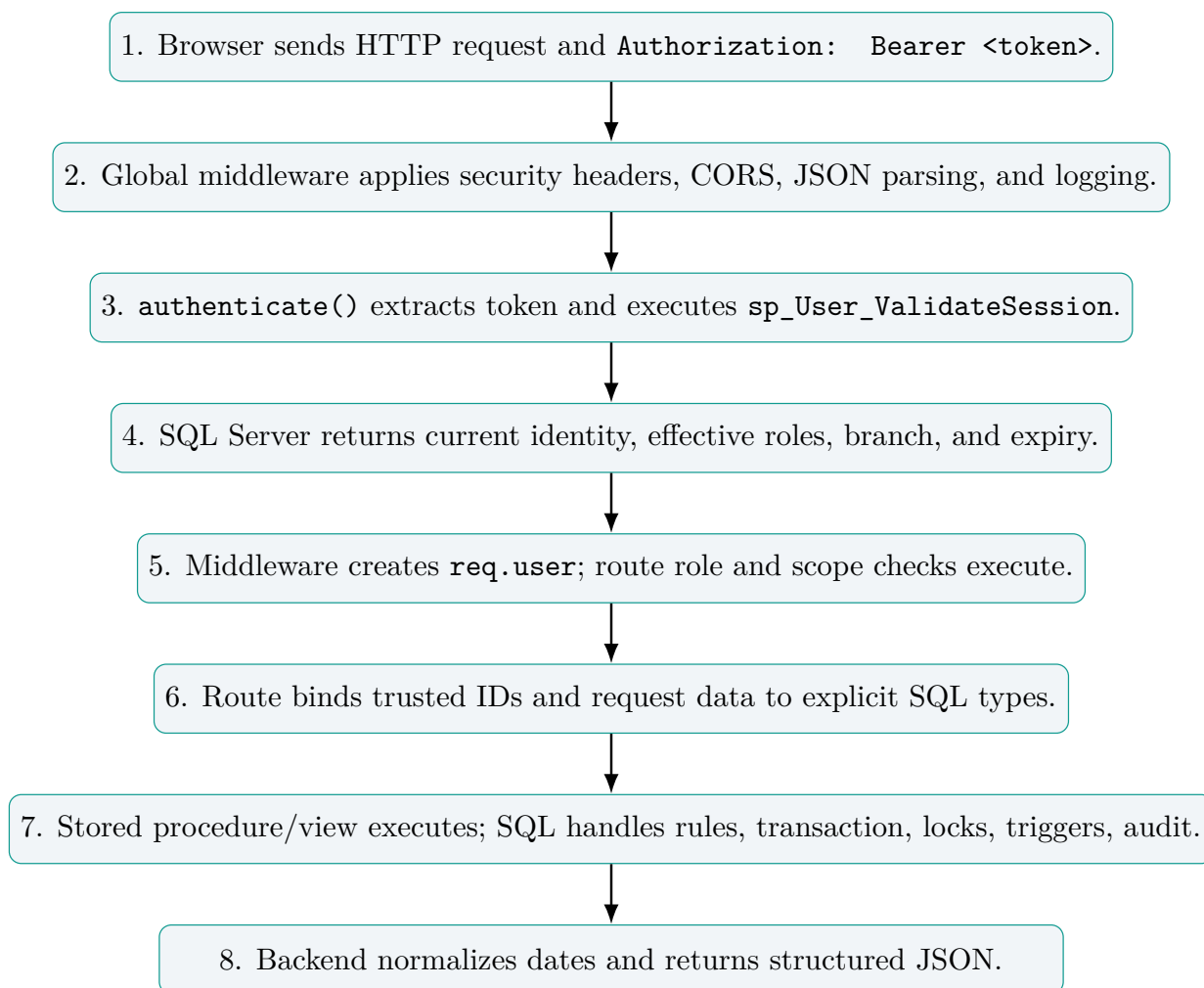


Figure 9.1: Protected API request lifecycle

9.2 Why identity is not accepted from the client

The client may submit business identifiers such as an account ID, but it is not trusted to submit the acting user ID. Routes add the validated identity after spreading request values:

```
1 values: {  
2   ...req.body,  
3   UserID: req.user.UserID  
4 }
```

Because `UserID` is assigned last, a malicious `UserID` in the JSON body is overwritten.

Chapter 10

Authentication

10.1 Three separate authentication contexts

Table 10.1: Authentication contexts

Context	Principal	Purpose
SQL connection	<code>BankAppLogin</code> , etc.	Authenticates the Node service to SQL Server.
Application login	Row in <code>dbo.Users</code>	Authenticates a bank-system person by password.
Session request	Opaque token in <code>dbo.Sessions</code>	Identifies an already authenticated application user.

10.2 Signup

The signup route validates required HTTP fields and calls `dbo.sp_User_SignUp`. The database checks uniqueness and creates the Customer/User/Customer-role relationship transactionally. The backend returns the created identifiers.

10.3 Login

`POST/api/auth/login` applies a rate limit of 20 attempts per 15 minutes and requires a password. It passes one optional identifier (username, user ID, customer ID, or employee ID), the password, and the configured session lifetime to `dbo.sp_User_Login`.

The database procedure:

1. validates that exactly one login identifier was supplied;
2. hashes the supplied password using `dbo.fn_HashPassword`;
3. finds an active user and active customer profile;
4. loads assigned roles and calculates effective roles;
5. validates HighAdmin/Employee/Admin identity consistency;
6. creates a random opaque token;
7. calculates `ExpiresAt = LoginTime + 10 minutes` by default;
8. inserts `dbo.Sessions`;
9. records a login audit entry;
10. returns IDs, roles, login time, token, and expiration.

10.4 Token design

The token is generated from random GUID values and stored server-side. It is not a JWT:

- no role is trusted from the token itself;
- the token is only a lookup key;
- revocation is immediate by setting `IsActive = 0`;
- role and branch changes become effective on the next validation request.

10.5 Session validation

Every protected router uses `authenticate()`. The middleware calls:

```
1 executeProcedure('dbo.sp_User_ValidateSession', {  
2     inputs: {  
3         SessionToken: TYPES.token,  
4         RequiredRole: TYPES.string50  
5     },  
6     values: {  
7         SessionToken: token,  
8         RequiredRole: requiredRole  
9     }  
10 });
```

The database validator checks:

- token existence and active session state;
- absolute expiration;
- active user and customer records;
- assigned Customer role;
- active Employee status for Employee privileges;
- manager title and `CanAccessAdmin` for Admin privileges;
- HighAdmin identity consistency;
- current EMPB branch assignment;
- an optional required role.

An expired session is deactivated, given a logout time, and audited as `SessionExpired`.

10.6 Creation of `req.user`

The validator record is attached to the request:

```
1 req.sessionToken = token;  
2 req.user = {  
3     ...user,  
4     roles: parseRoles(user.EffectiveRoles)  
5 };
```

A typical manager context contains:

```
1 {  
2     UserID: 25,  
3     CustomerID: 118,
```

```
4 EmployeeID: 42,  
5 CurrentBranchID: 2,  
6 EffectiveRoles: 'Customer,Employee,Admin',  
7 roles: ['Customer', 'Employee', 'Admin']  
8 }
```

10.7 Logout

Logout is itself protected. The token is first validated, then `dbo.sp_User_Logout` locks the active session row, marks it inactive, sets `LogoutTime`, and inserts a logout audit record.

Chapter 11

Authorization

11.1 Route-level role authorization

`authorize(...roles)` checks the parsed effective-role array:

```
1 const allowed = allowedRoles.some(  
2   (role) => req.user.roles.includes(role)  
3 );
```

A missing/invalid session produces HTTP 401. A valid identity without the required role produces HTTP 403.

11.2 Owner, branch, and global scope

`src/utils/access-scope.js` resolves three modes:

Table 11.1: Access-scope modes

Mode	Who	Filter
mine	Explicit personal scope or Customer	Force <code>CustomerID = req.user.CustomerID</code> .
branch	Effective Employee or Admin	Force <code>BranchID = req.user.CurrentBranchID</code> .
all	Effective HighAdmin	No customer/branch restriction.

11.3 Account ownership

Financial transaction routes call `assertAccountOwner`. This performs an owner-scoped account search before the financial procedure. The transaction procedure receives the authenticated `UserID` and checks ownership again. Therefore, staff visibility does not grant transaction ownership.

11.4 Employee scope

Employee routes implement additional target checks:

- normal Employee: own employee record only;
- Admin: current-branch employees only;
- HighAdmin: all employees.

For a manager, the backend forces `branchID` to the session branch and disables historical cross-branch search. The stored procedure independently enforces the business rule.

11.5 Loan scope

Loan listing uses the same personal/branch/global model. Loan creation forces non-HighAdmin staff to their current branch. Installment payment requires both the paying account and the loan borrower to match the authenticated customer.

11.6 Defense in depth

Authorization is deliberately duplicated

The backend blocks obviously invalid requests early and provides correct HTTP semantics. The database procedure performs the authoritative rule check inside the same transaction that changes data. Bypassing a frontend button or calling the API manually does not bypass the SQL rule.

Chapter 12

Business Procedure Communication

12.1 Route-to-procedure pattern

A route normally performs:

1. required-field validation;
2. authentication and role check;
3. scope/ownership precheck where applicable;
4. construction of an `inputs` map;
5. construction of a `values` object;
6. `executeProcedure`;
7. conversion of recordsets/output to a standard response.

12.2 Example: opening an account

```
1 executeProcedure('dbo.sp_Account_Open', {
2   inputs: {
3     UserID: TYPES.int,
4     BranchID: [TYPES.int, 'branchID'],
5     AccountTypeID: [TYPES.int, 'accountTypeID'],
6     InitialDeposit: [TYPES.money, 'initialDeposit']
7   },
8   outputs: {
9     AccountID: TYPES.int,
10    AccountNumber: TYPES.string30,
11    InitialDepositTransactionID: TYPES.int,
12    InitialDepositReadyToCompleteAt: TYPES.datetime
13  },
14  values: { ...req.body, UserID: req.user.UserID }
15 });
```

The route does not insert into `Account` directly. The procedure checks customer identity, branch and account type, creates the account, allocates an account number, optionally creates a pending initial-deposit transaction, and audits the operation.

12.3 Example: transfer

The backend confirms the authenticated user owns the source account, then calls `sp_Transaction_Transfer`. SQL Server validates amount, account states, destination, available balance, minimum balance, and ownership; it creates a pending transaction and determines `ReadyToCompleteAt`. Finalization later applies the debit/credit atomically and activates triggers/audit behavior.

12.4 Example: loan status

The backend first uses a parameterized view query to prove that the loan is in personal, branch, or global scope. It then calls `sp_Loan_GetStatus`, which returns separate loan and installment recordsets. The API preserves both and supplies date metadata by recordset.

Chapter 13

Reporting Communication

13.1 Report catalogue

The report route does not accept an arbitrary SQL view name. A client supplies a report key. The backend looks it up in the constant `REPORT_VIEWS`; the selected entry contains:

- fixed view name;
- fixed connection-pool name;
- allowed application roles;
- branch-filter strategy;
- fixed newest-first `ORDER BY`;
- date-field metadata.

13.2 Safe dynamic SQL

The query text contains an interpolated view and order expression, but both originate from server code, not request data. Client-controlled paging and branch values are still bound parameters:

```
1 SELECT R.*
2 FROM dbo.vw_AccountOperationalSummary AS R
3 WHERE R.BranchID = @BranchID
4 ORDER BY R.OpenDate DESC, R.AccountID DESC
5 OFFSET @Offset ROWS
6 FETCH NEXT @PageSize ROWS ONLY;
```

13.3 Branch filtering strategies

Table 13.1: Report branch-filter modes

Mode	Meaning
column	Filter the report view's <code>BranchID</code> .
account-link	Keep a transaction when either source or destination account belongs to the current branch.
customer-account-link	Keep a customer when the customer owns an account in the current branch.
No branch mode	Use the entire permitted view; typically HighAdmin or aggregate report.

13.4 Paging

`getPaging` clamps:

$$1 \leq page, \quad 1 \leq pageSize \leq 200$$

and calculates:

$$offset = (page - 1) \times pageSize.$$

This prevents unbounded report result pages.

Chapter 14

Database-Level Security

14.1 Technical SQL principals

The installation creates:

- BankAppLogin / BankAppUser
- BankReportLogin / BankReportUser
- BankAuditorLogin / BankAuditorUser
- BankHighAdminReportLogin / BankHighAdminReportUser
- BankMaintenanceLogin / BankMaintenanceUser

14.2 Database roles and privileges

Table 14.1: Database roles

Database role	Permission design
dbrole_bank_app_executor	Execute on the dbo schema plus selected operational views.
dbrole_bank_reporting	Select from controlled operational/aggregate views.
dbrole_bank_auditor	Select from audit, ledger, and selected oversight views.
dbrole_bank_highadmin_viewer	Select from all approved HighAdmin oversight views.
dbrole_bank_maintenance	Metadata visibility by default; scheduled jobs run in trusted SQL context.

The security script revokes direct table DML permissions from project roles. The application principal executes procedures; report principals read views.

14.3 Application roles are different

Application roles in `dbo.Roles` represent bank users. Database roles represent technical SQL principals. A HighAdmin application user does not connect to SQL Server as `BankHighAdminReportLogin`; the Node backend chooses the pool after validating the application session.

Chapter 15

SQL Injection Control

15.1 Bound data values

All normal request data is supplied through `request.input`. SQL Server receives it as a typed parameter, not executable SQL text. A value containing quotes or SQL keywords remains data.

15.2 Identifiers

Parameters cannot represent table/view/column names. The project therefore uses a server-side report whitelist. The browser can select only a known key; it cannot provide:

- an arbitrary view name;
- an arbitrary `ORDER BY` expression;
- a raw `WHERE` fragment;
- a stored-procedure name.

15.3 Static loan query

Loan listing uses a constant query string with parameters for `LoanID`, `CustomerID`, `BranchID`, `LoanStatus`, offset, and page size. No request text is concatenated into the SQL structure.

15.4 Stored procedures are not sufficient by themselves

Stored procedures are safe only when they also avoid unsafe dynamic SQL. The project combines stored procedures with parameter binding, controlled object names, and restricted SQL permissions.

Chapter 16

Proof That No ORM Is Used

16.1 Dependency evidence

The runtime dependencies are:

Listing 16.1: Runtime dependency set

```
1 cors
2 dotenv
3 express
4 express-rate-limit
5 helmet
6 morgan
7 mssql
```

There is no Sequelize, TypeORM, Prisma, Knex, Objection, Bookshelf, MikroORM, Drizzle, or comparable ORM dependency.

16.2 Connection evidence

src/config/db.js directly imports:

```
1 const sql = require('mssql');
```

and directly constructs:

```
1 new sql.ConnectionPool(config);
```

16.3 Execution evidence

src/utils/procedure.js directly uses:

```
1 request.input(name, type, value);
2 request.output(name, type);
3 request.execute(procedureName);
4 request.query(queryText);
```

16.4 Route evidence

Route files explicitly name:

- stored procedures, such as `dbo.sp_Account_Open`;
- SQL Server data types, such as `sql.Int` and `sql.Decimal(18,2)`;
- output parameters;
- recordsets and views.

16.5 Absent ORM artifacts

The source tree has no model/entity classes, no `schema.prisma`, no ORM migration format, no generated repository layer, no unit-of-work/change tracker, and no table-to-object association declarations.

Conclusion

The backend contains a small direct data-access adapter around the `mssql` driver. A helper that reduces repetitive driver calls is not an ORM. SQL Server objects and T-SQL remain explicit and visible at every route.

Chapter 17

Date and Time Handling

17.1 SQL driver configuration

The project uses SQL `DATETIME` values produced with local server time and sets `DB_USE_UTC=false` for localhost. This prevents the driver from interpreting local SQL timestamps as UTC instants.

17.2 Date-only versus date-time

`src/utils/http.js` distinguishes:

- calendar fields such as `BirthDate`, `OpenDate`, `DueDate`; returned as `YYYY-MM-DD`;
- timestamp fields such as `TransactionDate`, `ReadyToCompleteAt`, `ActionDate`, `ExpiresAt`; returned as ISO timestamps.

This prevents calendar dates from shifting by one day in a browser timezone.

17.3 Response metadata

Routes and reports include `dateFields` or `dateFieldsByRecordset`. The frontend uses these declarations instead of guessing every column type.

Chapter 18

HTTP Responses and Error Handling

18.1 Success response

The normal envelope is:

```
1 {  
2   success: true,  
3   data: ...,  
4   output: ...,  
5   meta: ...  
6 }
```

Creation routes use HTTP 201; ordinary successful routes use HTTP 200.

18.2 Errors

Table 18.1: HTTP error behavior

Status	Meaning in the backend
400	Missing/invalid input or a SQL business-rule error.
401	Missing, invalid, inactive, or expired session.
403	Valid identity but insufficient role, ownership, or branch scope.
404	Unknown API route, report key, or explicitly missing resource.
500	Unexpected non-SQL server failure.

SQL error details are normalized to message, number, state, class, procedure, and line number. Development responses include a stack trace; production responses do not.

Known semantic limitation

The current central handler maps most SQL errors to HTTP 400, including some connection or permission failures that could more precisely be HTTP 500/503. This does not bypass security, but production error classification could be improved.

Chapter 19

Transactions, Concurrency, and Atomicity

19.1 Backend versus database transactions

The Node routes do not attempt to coordinate multi-step banking writes in JavaScript. The relevant stored procedure owns the SQL transaction. This keeps locks, validation, data modification, audit insertion, and trigger effects in one SQL Server transaction.

19.2 Pending financial transactions

Deposit, withdrawal, transfer, and installment payment can create **Pending** records. The API returns a transaction ID and **ReadyToCompleteAt**. The finalizer or batch job later applies posted balance changes after rechecking account status, available balance, and other rules.

19.3 Locks

Business procedures use SQL locking patterns such as **UPDLOCK** and **HOLDLOCK** where concurrent updates could otherwise violate ownership, balance, assignment, or state-machine rules. The backend does not simulate these locks.

19.4 Triggers

Balance updates can fire the **BranchLedger** synchronization trigger; installment-related transactions can fire installment status synchronization. Because triggers execute within the originating SQL transaction, a trigger failure rolls back the originating change.

Chapter 20

Audit and Branch Ledger

20.1 Audit ownership

Authentication and business procedures insert into `dbo.AuditLog`. The backend does not construct audit rows from client descriptions. This ensures the audit event is committed or rolled back with the authoritative operation.

20.2 Session audit

Login, logout, and expiration create explicit audit actions. Routine session validation intentionally does not insert an audit row for every API call, avoiding excessive noise.

20.3 Branch ledger

Account balance changes automatically update the branch cached balance and insert `BranchLedger` records through a database trigger. The report endpoint reads `vw_BranchLedgerReport`; refreshing the report does not update the ledger.

20.4 Safe audit reporting

The HighAdmin audit endpoint uses the dedicated audit pool and `vw_AuditTrail_Safe`. This separates application execution permission from read-only oversight permission.

Chapter 21

Scheduled Jobs and Manual Maintenance

21.1 SQL Server Agent

Three scheduled jobs call maintenance procedures:

- process pending transactions;
- process overdue loan installments;
- apply monthly account interest.

They do not have browser bearer tokens. They execute as trusted database maintenance operations.

21.2 Manual API equivalents

Admin/HighAdmin endpoints can invoke the same maintenance behavior manually. In that path, the API session and role are validated first. Branch-manager overdue processing is restricted to the manager's current branch; HighAdmin may process one branch or all branches.

Chapter 22

Detailed Communication Scenarios

22.1 Scenario 1: login

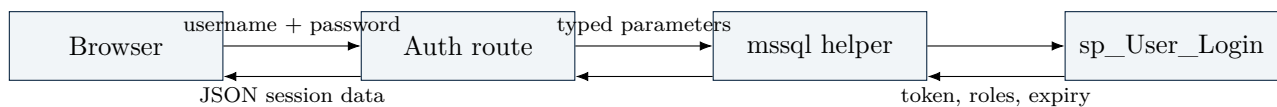


Figure 22.1: Login communication sequence

22.2 Scenario 2: personal transfer

1. Browser sends source account, destination account, amount, and bearer token.
2. Session validation reconstructs the current customer.
3. `assertAccountOwner` calls `sp_Account_Search` in personal scope.
4. Route calls `sp_Transaction_Transfer` with trusted `UserID`.
5. Procedure validates source ownership again, account states, amount, destination, available balance, and minimum balance.
6. Procedure inserts a pending transaction and audit information.
7. API returns transaction ID and ready timestamp.
8. Finalizer/Agent later posts both balance changes in one SQL transaction.

22.3 Scenario 3: manager views branch accounts

1. Validator returns the manager's current `BranchID`.
2. `resolveAccessScope` selects branch mode.
3. The backend overwrites any requested branch filter with the session branch.
4. `sp_Account_Search` receives both `UserID` and forced `BranchID`.
5. SQL validates effective role and branch relation and returns matching rows.

22.4 Scenario 4: HighAdmin audit report

1. `authenticate()` validates the session and effective HighAdmin role.
2. The report key maps to `vw_AuditTrail_Safe`, pool `audit`, and newest-first ordering.

3. Paging values are parameterized.
4. The audit SQL login can select the safe view but has no general table DML permission.
5. Date metadata marks `ActionDate` as a timestamp.

22.5 Scenario 5: session expires

1. Browser sends an old token.
2. `sp_User_ValidateSession` finds `ExpiresAt <= GETDATE()`.
3. SQL sets `IsActive=0`, sets `LogoutTime`, inserts `SessionExpired`, and raises an error.
4. Authentication middleware returns HTTP 401.
5. Frontend clears its stored session and redirects to login.

Chapter 23

Testing and Diagnostics

23.1 Syntax checks

The package includes:

```
1 npm run check
```

which runs Node syntax validation across the backend source.

23.2 Health and connection diagnosis

`/api/health` distinguishes an HTTP problem from a SQL connection problem and reveals the actual database/login context without exposing credentials.

23.3 Postman testing

The recorded comprehensive collection contains 108 requests and exercises health, authentication states, role boundaries, sample-data verification, expected SQL failures, all reports, write workflows, and maintenance operations. A recorded run produced 457 passing and 9 failing assertions out of 466; the remaining failures were traced to repeat-run state mutation in optional workflows rather than broad authentication, report-pool, or core access failures.

23.4 Useful diagnostic distinctions

- 401: inspect bearer token, session state, and expiration.
- 403: inspect effective role, current branch, and ownership scope.
- SQL login failure: inspect the selected pool credentials and database mapping.
- Report 400 with core APIs working: inspect report/audit/highadmin pool credentials and view grants.
- Wrong local time: inspect DB_USE_UTC, SQL server clock, and date metadata.

Chapter 24

Security and Production Hardening

24.1 Existing controls

- Helmet security headers.
- CORS allow-list.
- JSON body-size limit.
- Login rate limiting.
- Opaque server-side sessions with fixed expiry and revocation.
- Effective-role recalculation per protected request.
- Owner/branch/global scopes.
- Typed SQL parameter binding.
- Controlled report identifiers.
- Separate reporting/audit SQL identities.
- No direct project-role table DML.
- Database transactions, locks, triggers, constraints, and audit.

24.2 Recommended production improvements

- Serve API and frontend over HTTPS.
- Replace deterministic unsalted SHA-256 password storage with a slow salted password KDF such as Argon2id, bcrypt, or PBKDF2.
- Store session tokens only as hashes in the database.
- Prefer Secure, HttpOnly, SameSite cookies when deployment architecture permits.
- Add account lockout/backoff and credential-rotation procedures.
- Separate operational SQL errors from infrastructure errors in HTTP status mapping.
- Add structured request correlation IDs and centralized logging.
- Add automated integration tests against an isolated resettable database.
- Configure dedicated report/audit credentials instead of fallback.
- Add scheduled cleanup of old inactive session rows.

Chapter 25

Conclusion

The backend is a controlled translation layer between HTTP and SQL Server. It does not replace database logic with JavaScript models. Express provides API mechanics and early security checks; `mssql` provides typed connectivity; SQL Server procedures and views remain explicit; and the database owns the state transitions and transactional rules. Authentication is collaborative: the backend handles HTTP presentation and middleware, while the database verifies credentials, creates and validates sessions, calculates effective roles, and enforces ownership and branch rules. The use of named connection pools and restricted database roles separates operational execution from reporting and auditing.

Implementation summary

The browser never talks to SQL Server. The backend never trusts client-supplied identity or role values. Every protected request validates an opaque server-side token, reconstructs current privileges from SQL Server, binds values to explicit SQL types, and calls a named procedure or whitelisted view. The absence of ORM dependencies, models, migrations, and generated queries—combined with direct `mssql` calls and explicit stored-procedure names—proves that the project uses direct SQL data access rather than an ORM.

Chapter A

Complete API-to-SQL Inventory

Table A.1: Complete endpoint communication inventory

Method	Endpoint	Application access	SQL target	Communication note
System				
GET	/api/health	Public	Direct parameter-free diagnostic SELECT	Checks database name, SQL login, and database user.
Setup				
POST	/api/setup/initial-high-admin	Public; one-time DB rule	dbo.sp_HighAdmin_CreateInitialUser	Creates the first HighAdmin; database rejects later attempts.
Authentication				
POST	/api/auth/signup	Public	dbo.sp_User_SignUp	Creates a customer application user.
POST	/api/auth/login	Public; rate-limited	dbo.sp_User_Login	Verifies credentials and creates a fixed-lifetime session.
POST	/api/auth/logout	Authenticated	dbo.sp_User_ValidateSession → dbo.sp_User_Logout	Validates then revokes the current session.
GET	/api/auth/me	Authenticated	dbo.sp_User_ValidateSession	Returns the identity context attached to req.user.
Customers				
GET	/api/customers	Employee/Admin branch; HighAdmin all	dbo.sp_Customer_Search	Route precheck plus database branch scoping through customer accounts.
POST	/api/customers	Employee, Admin, HighAdmin	dbo.sp_Customer_Create	Creates a customer record.
PUT	/api/customers/:customerID	Employee, Admin, HighAdmin	dbo.sp_Customer_Update	Procedure rechecks scope and rules.
DELETE	/api/customers/:customerID	Employee, Admin, HighAdmin	dbo.sp_Customer_Delete	Soft deactivation; database blocks unsafe deactivation.
Accounts				
GET	/api/accounts/mine	Authenticated owner scope	dbo.sp_Account_Search	For personal accounts of any application user.
GET	/api/accounts/options/account-types	Authenticated	dbo.sp_AccountType_ListOptions	Safe dropdown catalogue.
GET	/api/accounts	Customer own; staff branch; HighAdmin all	dbo.sp_Account_Search	Scope is resolved in access-scope.js.

Method	Endpoint	Application access	SQL target	Communication note
GET	/api/accounts/:accountID	Scope-dependent	dbo.sp_Account_Search → dbo.sp_Account_GetInfo	Visibility precheck then detail procedure.
GET	/api/accounts/:accountID/history	Scope-dependent	dbo.sp_Account_Search → dbo.sp_Transaction_GetAccountHistory	Visibility precheck then history procedure.
POST	/api/accounts	Authenticated	dbo.sp_Account_Open	Self-service account opening for current customer identity.
POST	/api/accounts/:accountID/close	Employee, Admin, HighAdmin; visible account	dbo.sp_Account_Close	Lifecycle management, not a financial transaction.
POST	/api/accounts/:accountID/freeze	Employee, Admin, HighAdmin; visible account	dbo.sp_Account_Freeze	Branch/all scope enforced.
POST	/api/accounts/:accountID/unfreeze	Admin, HighAdmin; visible account	dbo.sp_Account_Unfreeze	Normal employees cannot unfreeze.
POST	/api/accounts/:accountID/change-type	Employee, Admin, HighAdmin; visible account	dbo.sp_Account_ChangeType	Database checks pending transactions and minimum balance.
POST	/api/accounts/maintenance/dormant-sweep	Admin, HighAdmin	dbo.sp_Account_DormantSweep	Manual maintenance entry point.
POST	/api/accounts/maintenance/apply-monthly-interest	Admin, HighAdmin	dbo.sp_Account_ApplyMonthlyInterest	Manual equivalent of the Agent operation.
Transactions				
POST	/api/transactions/deposit	Authenticated account owner	dbo.sp_Account_Search → dbo.sp_Transaction_Deposit	Owner precheck in backend; ownership rechecked in SQL.
POST	/api/transactions/withdraw	Authenticated account owner	dbo.sp_Account_Search → dbo.sp_Transaction-Withdrawal	Creates a pending transaction when valid.
POST	/api/transactions/transfer	Authenticated source-account owner	dbo.sp_Account_Search → dbo.sp_Transaction_Transfer	Destination rules and balances are checked in SQL.
POST	/api/transactions/:transactionID/finalize	Authenticated transaction owner	dbo.sp_Transaction_Finalize	Advances one transaction when ready.
POST	/api/transactions/:transactionID/reverse	Authenticated transaction owner	dbo.sp_Transaction_Reverse	Cancels pending or reverses completed transaction under DB rules.
POST	/api/transactions/maintenance/process-pending-batch	Admin, HighAdmin	dbo.sp_Transaction_ProcessPendingBatch	Manual batch processing.

Loans

Method	Endpoint	Application access	SQL target	Communication note
GET	/api/loans	Customer own; staff branch; HighAdmin all	Parameterized SELECT from dbo.vw_LoanOperationalSummary	Static SQL with bound filter and paging values.
POST	/api/loans	Employee, Admin, HighAdmin	dbo.sp_Loan_Create	Non-HighAdmin branch ID is forced to current branch.
GET	/api/loans/:loanID/status	Scope-dependent	Parameterizedviewprecheck → dbo.sp_Loan_GetStatus	Returns loan summary and installment recordsets.
POST	/api/loans/installments/:installmentID/pay	Authenticated borrower/account owner	dbo.sp_Account_Search → dbo.sp_Loan_PayInstallment	Both loan borrower and payment-account ownership are enforced.
POST	/api/loans/maintenance/process-overdue-installments	Admin branch; HighAdmin optional branch/all	dbo.sp_Loan_ProcessOverdueInstallments	Manual overdue processing.
Employees				
GET	/api/employees	Employee self; Admin branch; HighAdmin all	dbo.sp_Employee_Search	Backend forces employee or branch scope.
GET	/api/employees/:employeeID	Employee self; Admin branch; HighAdmin all	dbo.sp_Employee_Search → dbo.sp_Employee_GetInfo	Target visibility is checked before details.
POST	/api/employees	Admin only	dbo.sp_Employee_Hire	Ordinary hiring into the manager's current branch.
POST	/api/employees/:employeeID/create-user-account	Admin branch; HighAdmin	dbo.sp_Employee_Search → dbo.sp_Employee_CreateUserAccount	Creates Customer/User/roles for an employee login.
POST	/api/employees/:employeeID/change-job-title	Admin branch; HighAdmin	dbo.sp_Employee_Search → dbo.sp_Employee_ChangeJobTitle	Manager-level governance remains HighAdmin-specific.
POST	/api/employees/:employeeID/fire	Admin branch; HighAdmin	dbo.sp_Employee_Search → dbo.sp_Employee_Fire	Ends employment assignment under DB rules.
POST	/api/employees/:employeeID/suspend	Admin branch; HighAdmin	dbo.sp_Employee_Search → dbo.sp_Employee_Suspend	Removes effective staff privileges while preserving customer access.
GET	/api/employees/:employeeID/branch-history	Employee self; Admin branch; HighAdmin all	dbo.sp_Employee_Search → dbo.sp_EMPBranchHistory_Get	Historical assignment visibility is controlled.
Branches				
GET	/api/branches/options	Authenticated	dbo.sp_Branch_ListOptions	Returns only non-sensitive branch identifiers for dropdowns.
GET	/api/branches	Employee, Admin, HighAdmin	dbo.sp_Branch_GetInfo	DB determines visible branch information.

Method	Endpoint	Application access	SQL target	Communication note
GET	/api/branches/:branchID	Employee, Admin, HighAdmin	dbo.sp_Branch_GetInfo	Single branch details.
HighAdmin				
POST	/api/highadmin/managers	HighAdmin	dbo.sp_HighAdmin_HireManager	Branch ID and manager title are validated before SQL call.
POST	/api/highadmin/managers/:employeeID/promote	HighAdmin	dbo.sp_HighAdmin_PromoteToManager	Promotes and assigns a manager to a branch.
POST	/api/highadmin/managers/:employeeID/downgrade	HighAdmin	dbo.sp_HighAdmin_DowngradeManager	Removes manager-level role under transactional rules.
POST	/api/highadmin/managers/:employeeID/fire	HighAdmin	dbo.sp_HighAdmin_FireManager	Manager termination.
POST	/api/highadmin/managers/:employeeID/suspend	HighAdmin	dbo.sp_HighAdmin_SuspendManager	Manager suspension.
POST	/api/highadmin/branches/:branchID/replace-manager	HighAdmin	dbo.sp_HighAdmin_ReplaceBranchManager	Atomic manager replacement.
Employee transfers				
POST	/api/employee-transfers/request-by-employee	Employee, Admin	dbo.sp_EmployeeTransfer_RequestByEmployee	Creates a transfer request for the current employee.
POST	/api/employee-transfers/request-by-manager	Admin, HighAdmin	dbo.sp_EmployeeTransfer_RequestByManager	Manager initiates a transfer for an employee.
POST	/api/employee-transfers/:id/current-manager-decision	Admin	dbo.sp_EmployeeTransfer_ApproveCurrentManager	First approval stage.
POST	/api/employee-transfers/:id/destination-manager-decision	Admin	dbo.sp_EmployeeTransfer_ApproveDestinationManager	Second stage and final branch assignment update.
Reports				
GET	/api/reports	Authenticated	In-memory REPORT_VIEWS catalogue	Lists report keys and metadata; session validation still runs.
GET	/api/reports/:reportKey	Report-specific roles	Parameterized SELECT from whitelisted reporting view	Hard-coded view/order configuration, branch filtering, and paging.

Chapter B

Report Catalogue

Table B.1: Whitelisted API reports

Report definition	Application access and scope	Ordering and date metadata
Key: customer-basic-profile View: vw_CustomerBasicProfile Pool: report	Roles: Employee/Admin/HighAdmin Staff scope: Customer linked to an account in current branch	Order: RegistrationDate DESC Dates: BirthDate: date; RegistrationDate: date
Key: customer-account-summary View: vw_CustomerAccountSummary Pool: report	Roles: Employee/Admin/HighAdmin Staff scope: BranchID column	Order: OpenDate DESC Dates: OpenDate: date; CloseDate: date
Key: account-operational-summary View: vw_AccountOperationalSummary Pool: report	Roles: Employee/Admin/HighAdmin Staff scope: BranchID column	Order: OpenDate DESC Dates: OpenDate: date; CloseDate: date
Key: employee-directory View: vw_EmployeeDirectory Pool: report	Roles: Employee/Admin/HighAdmin Staff scope: BranchID column	Order: HireDate DESC Dates: HireDate: date; CurrentBranchStartDate: date
Key: pending-transactions View: vw_PendingTransactions Pool: report	Roles: Employee/Admin/HighAdmin Staff scope: From/to account linked to branch	Order: TransactionDate DESC Dates: TransactionDate: datetime; ReadyToCompleteAt: datetime
Key: loan-operational-summary View: vw_LoanOperationalSummary Pool: report	Roles: Employee/Admin/HighAdmin Staff scope: BranchID column	Order: StartDate DESC Dates: StartDate: date; EndDate: date
Key: daily-transaction-summary View: vw_DailyTransactionSummary Pool: report	Roles: Admin/HighAdmin Staff scope: Not branch-filtered in route	Order: TransactionDay DESC Dates: TransactionDay: date
Key: account-status-summary View: vw_AccountStatusSummary Pool: report	Roles: Admin/HighAdmin Staff scope: BranchID column	Order: BranchID DESC Dates: None
Key: loan-overdue-summary View: vw_LoanOverdueSummary Pool: report	Roles: Admin/HighAdmin Staff scope: BranchID column	Order: OldestOverdueDueDate DESC Dates: OldestOverdueDueDate: date
Key: highadmin-employee-branch-overview View: vw_HighAdmin_EmployeeBranchOverview Pool: highadminReport	Roles: HighAdmin Staff scope: All	Order: StartDate DESC Dates: HireDate, StartDate, EndDate: date

Report definition	Application access and scope	Ordering and date metadata
Key: highadmin-branch-financial-overview View: vw_HighAdmin_BranchFinancialOverview Pool: highadminReport	Roles: HighAdmin Staff scope: All	Order: BranchID DESC Dates: None
Key: highadmin-user-access-overview View: vw_HighAdmin_UserAccessOverview Pool: highadminReport	Roles: HighAdmin Staff scope: All	Order: UserID DESC Dates: None
Key: audit-trail View: vw_AuditTrail_Safe Pool: audit	Roles: HighAdmin Staff scope: All	Order: ActionDate DESC Dates: ActionDate: datetime
Key: branch-ledger View: vw_BranchLedgerReport Pool: audit	Roles: HighAdmin Staff scope: All	Order: EntryDate DESC Dates: EntryDate: datetime

Chapter C

Stored Procedures Called by the Backend

The following procedures are named directly in the Node.js source. Procedures used internally by other SQL procedures may not appear in this list.

Table C.1: Directly called stored procedures

Procedure	Backend use
dbo.sp_AccountType_ListOptions	Safe account-product dropdown.
dbo.sp_Account_ApplyMonthlyInterest	Apply eligible monthly interest.
dbo.sp_Account_ChangeType	Change account product.
dbo.sp_Account_Close	Close account.
dbo.sp_Account_DormantSweep	Mark inactive accounts dormant.
dbo.sp_Account_Freeze	Freeze account.
dbo.sp_Account_GetInfo	Account detail.
dbo.sp_Account_Open	Account creation and optional initial-deposit transaction.
dbo.sp_Account_Search	Personal/branch/global account search and visibility prechecks.
dbo.sp_Account_Unfreeze	Unfreeze account.
dbo.sp_Branch_GetInfo	Branch information.
dbo.sp_Branch_ListOptions	Safe branch dropdown.
dbo.sp_Customer_Create	Create customer.
dbo.sp_Customer_Delete	Soft deactivate scoped customer.
dbo.sp_Customer_Search	Branch/all customer directory search.
dbo.sp_Customer_Update	Update scoped customer.
dbo.sp_EMPBranchHistory_Get	Temporal branch assignment history.
dbo.sp_EmployeeTransfer_ApproveCurrentManager	Origin-manager decision.
dbo.sp_EmployeeTransfer_ApproveDestinationManager	Destination-manager decision and completion.
dbo.sp_EmployeeTransfer_RequestByEmployee	Employee transfer request.
dbo.sp_EmployeeTransfer_RequestByManager	Manager transfer request.
dbo.sp_Employee_ChangeJobTitle	Change ordinary employee title.
dbo.sp_Employee_CreateUserAccount	Create employee customer/login roles.
dbo.sp_Employee_Fire	Terminate ordinary employee.
dbo.sp_Employee_GetInfo	Employee details.
dbo.sp_Employee_Hire	Manager hires ordinary employee.
dbo.sp_Employee_Search	Employee self/branch/all search.
dbo.sp_Employee_Suspend	Set employee on leave/suspended.
dbo.sp_HighAdmin_CreateInitialUser	One-time first HighAdmin creation.
dbo.sp_HighAdmin_DowngradeManager	Downgrade manager.

Procedure	Backend use
dbo.sp_HighAdmin_FireManager	Terminate manager.
dbo.sp_HighAdmin_HireManager	Atomic manager/vice-manager hire.
dbo.sp_HighAdmin_PromoteToManager	Promote employee to manager.
dbo.sp_HighAdmin_ReplaceBranchManager	Atomic branch-manager replacement.
dbo.sp_HighAdmin_SuspendManager	Suspend manager.
dbo.sp_Loan_Create	Create loan and installment schedule.
dbo.sp_Loan_GetStatus	Return loan and installment status.
dbo.sp_Loan_PayInstallment	Create installment-payment transaction.
dbo.sp_Loan_ProcessOverdueInstallments	Update overdue/default states.
dbo.sp_Transaction_Deposit	Create deposit transaction.
dbo.sp_Transaction_Finalize	Finalize one pending transaction.
dbo.sp_Transaction_GetAccountHistory	Account transaction history.
dbo.sp_Transaction_ProcessPendingBatch	Batch-finalize ready transactions.
dbo.sp_Transaction_Reverse	Cancel/reverse transaction.
dbo.sp_Transaction_Transfer	Create transfer transaction.
dbo.sp_Transaction-Withdrawal	Create withdrawal transaction.
dbo.sp_User_Login	Credential verification and session creation.
dbo.sp_User_Logout	Session revocation.
dbo.sp_User_SignUp	Customer application signup.
dbo.sp_User_ValidateSession	Protected-request identity, role, branch, and expiration validation.